Day 9 Revision Sheet

Oct 28, 2026 (finished Oct 29) | Python: list len and looping, average, second largest, recall of Oct 20 and 21 | Git: remote revise | Smooth

1. Lists: len() and looping

- `len(marks)` = number of items. The last index is `len(marks) - 1`, so `range(len(marks))` stops there (stop is excluded, same as slicing). For a STRING, len counts characters, so `len(" ")` is 1.
- **By value:** `for m in marks:` gives each item's VALUE (72, then 45, then 88). **By index:** `for i in range(len(marks)):` gives 0, 1, 2 and you read `marks[i]`. Need both? Use `for i, m in enumerate(marks):` (cleaner than range(len(...))).
- **Why** `n = n * 10` **does not change the list:** `n` is only a name that points at one item's value on each pass. The line points `n` at a NEW number (10, 20, 30). The list is never touched, so `print(nums)` still shows `[1, 2, 3]`.
- **What does change the items:** assign by index. `for i in range(len(nums)): nums[i] = nums[i] * 10` -> `[10, 20, 30]`. Here `len` only supplies the stop value; the change is `nums[i] = ...`.
- **Rebinding vs mutating:** rebinding the loop variable never changes the list, but calling a method on a mutable item does (in a list of lists, `n.append(9)` changes the original).
- Excel bridge: `for m in marks` reads down the column cell by cell; `marks[i]` is like `INDEX(range, i+1)` because Python counts from 0.

2. Practice problems and what they taught

2.1 Average of marks

- `marks = [40,70,90,80,80]`; `average = sum(marks) / len(marks)` -> 360 / 5 = 72.0. `/` always returns a float.
- An empty list gives `ZeroDivisionError: division by zero` (0 / 0). Guard it with `if len(marks) > 0:` when the list can be empty.

2.2 Second largest distinct number

- Steps: `unique = list(set(nums))` (the set drops duplicates; a set has no order and no `.sort()`, so convert back to a list), then `unique.sort()`, then `unique[-2]`.
- **Silent bug I wrote first:** `unique.sort` without brackets. It only names the method, never calls it, and Python shows no error. The list stayed unsorted (I saw `[70, 50, 30]`), and the answer 50 was right only by luck. On other lists the same code returned wrong answers. Missing `()` is the classic silent bug (same for `nums.reverse`, `x.append`).
- **Guard:** `if len(unique) >= 2:` (at least 2 distinct values). `[5,5,5]`, `[]` and `[7]` go to the else branch. Without the guard, `[5,5,5]` raises `IndexError: list index out of range`. With negatives it still works: `[-4,-9,-1]` -> -4.
- Interview edge: the one-pass version uses two variables (largest and second) and no sorting. It is the max pattern with one extra variable.

3. Recall fixes (Oct 20 and Oct 21)

Topic	Fact and reason
`"5" + 3`	TypeError: can only concatenate str (not "int") to str. + cannot mix str and int. Fix: `int("5") + 3` = 8 or `"5" + str(3)` = "53".
`"5" * 3`	'555' (a STRING, type str). * on a string repeats it.
`7 // 2`, `7 % 2`, `7 / 2`	3, 1, 3.5. `//` rounds DOWN (toward negative infinity): `-7 // 2` is -4, not -3. `%` is the remainder. `/` is always a float.
`x = input()` then `x + 1`	x is a str (input always returns str). `x + 1` is a TypeError because + cannot mix str and int. Fix: `int(x) + 1`. This is NOT because of immutability.
`s[0] = "j"`	TypeError: 'str' object does not support item assignment. Cause: strings are immutable.
`len("")` vs `len(" ")`	0 and 1. A space is a character. `""` is falsy and `" "` is truthy, so an input of only spaces passes `if text:`. Check with `if not text.strip():`.

4. Git: the remote step

- Chain practised: `git init`, `git branch -M main`, create a file, `git add practice.py`, `git commit -m "Added average marks calculation"`, `git remote add origin <url>`, `git remote -v`, `git push -u origin main`. Status after each: No commits yet -> untracked -> Changes to be committed (new file) -> working tree clean.
- **origin is the nickname (remote name) for the GitHub URL.** Skip `git remote add origin` and `git push origin main` fails with 'origin does not appear to be a git repository'. It is a convention, so you could name it `backup`.
- **main vs master is a different thing:** it is the BRANCH name. Git may create `master` on `git init`, GitHub's default is `main`, so `git branch -M main` renames it to match. Mismatch error looks like `src refspec main does not match any`.
- `git remote -v` prints two lines for origin, one (fetch) and one (push). On a fresh Colab runtime `git commit` can fail with 'Author identity unknown': set `git config --global user.name` and `user.email` first.
- Push from Colab needs a personal access token, not a password. I did not retest the push today (it was proven earlier on git-practice), so that line stays untested for Oct 28.

5. My mistakes and the fix

What I said / did	Correct version
`x + 1` on input fails 'because string is immutable'	It fails because + cannot mix str and int. Immutability is a different idea (changing characters in place).
'Type error' for `s[0]='j'`	Name it exactly: TypeError: 'str' object does not support item assignment.
`//` 'removes the decimal part'	It rounds DOWN. `-7 // 2` is -4. Chopping decimals would give -3.
`len` explained as 'used to check empty spaces'	A space is a character: `len(" ")` is 1. `""` is falsy, `" "` is truthy; use `.strip()`.
To change list items: 'use the len function'	len only counts. Assign by index: `nums[i] = nums[i] * 10`.
`unique.sort` with no brackets	No call, no sorting, no error. Write `unique.sort()`. My answer was right by luck.
Guard worded as 'more than 2'	`>= 2` means at least 2 distinct values.
origin 'needed because git says master and GitHub says main'	origin = the remote's nickname. main/master = the branch name. Two separate things.
'len also includes hidden space' (in a LIST recall)	That is a string fact. For a list, len counts items.

6. How I improved

- All four answers in the loop/len question were right on the first try, with the trace written line by line.
- I fixed the silent bug, then wrote an else branch that covers the all-equal, empty and one-item lists, and tested my own prediction of 61 before running.
- I named origin as my hardest Git step honestly, then separated it from the branch name.

7. Practice to retry (no solutions)

1. Average of `scores`, but print 'empty list' instead of crashing when `scores = []`. Test with both a full and an empty list.
2. Find the largest and the second largest distinct number in ONE pass, without sorting. Test with `[4, 9, 9, 2]`.
3. `nums = [4, 7, 9]`: double every item IN PLACE. Then build a NEW list of doubled items and leave the original unchanged. Print both.
4. Predict, then run: `x = ['a','b','c']`, `for m in x: m = m + '!'`, then `print(x)`. Say why.
5. Third smallest distinct number in a list, with a guard for lists that are too short.
6. Predict: `print('7' + '3')`, `print('7' * 3)`, `print(7 + 3)`, `print(-9 // 4, -9 % 4)`. Give the reason for each.
7. A teammate types `git push orgin main` (typo). Say what Git replies and why, and what `git remote -v` would show.

8. Recall questions (answer from memory)

1. What does `n = n * 10` do inside `for n in nums`, and how do you actually change the items?
2. What does `unique.sort` (no brackets) do, and why is it dangerous?
3. Why is a guard like `len(unique) >= 2` needed for second largest?
4. Why does `x + 1` fail when x comes from input()? Name the exact error.
5. What does `//` do with negative numbers? Give an example.
6. What is `origin`, and how is it different from `main`?
7. What is the difference between `len('')` and `len(' ')`, and how do you detect a blank input?

9. Open items

- Re-test weak spots: silent missing brackets, exact error names, blending two reasons, `//` rounding, origin vs main.
- `git push` from Colab was not retested on Oct 28.
- Commit the Day 9 README entry as its own commit with a message that says what changed.
- Oct 29 (Day 10) tasks are next.